

选择：单选、多选

简答：

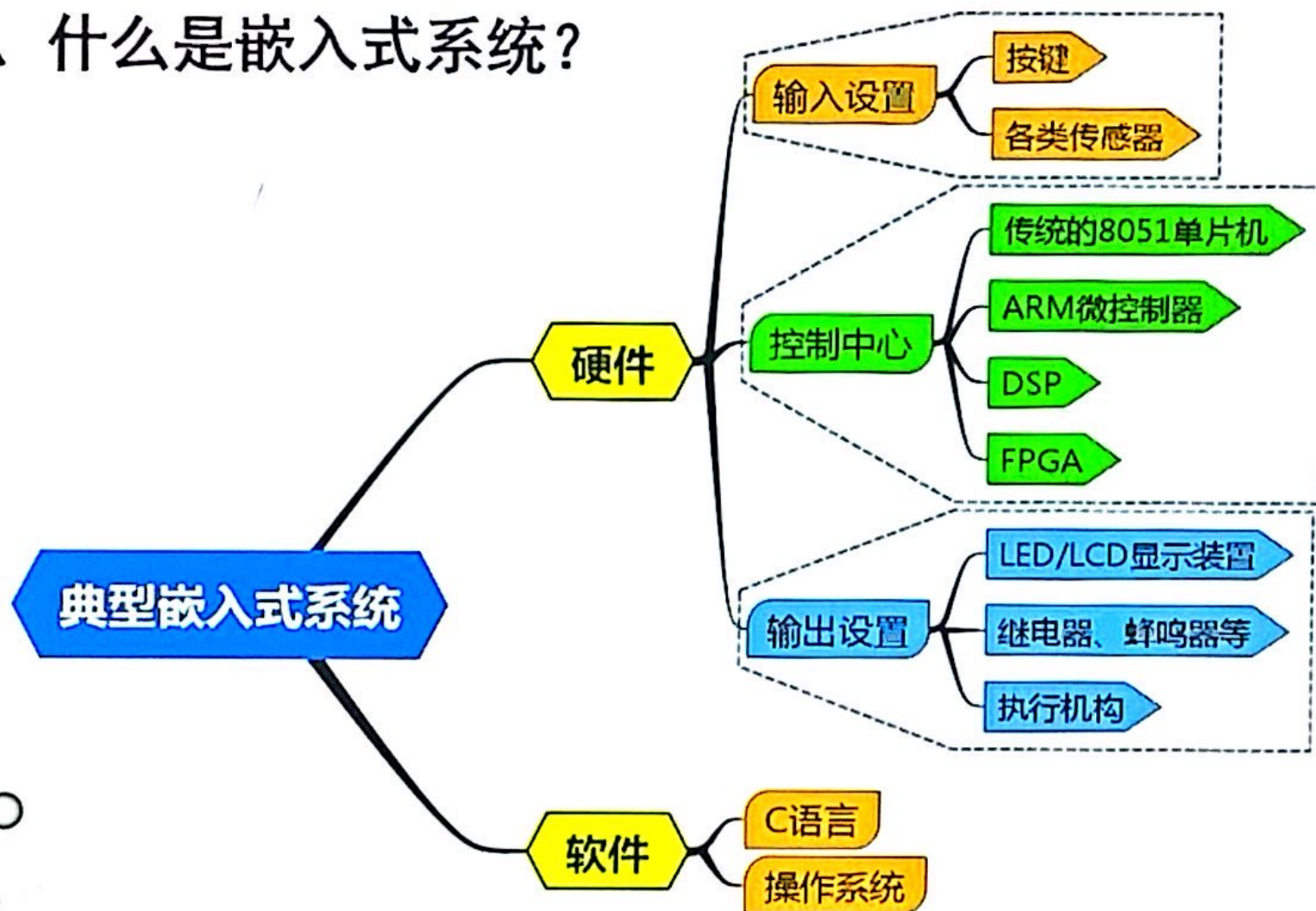
读程序

综合性应用

计算分析题



1、什么是嵌入式系统？



1、什么是嵌入式系统？

嵌入式系统

以应用为中心

以计算机技术为基础

软硬件可裁剪

满足应用系统对功能、可靠性、成本、体积和功耗等严格要求的专用计算机系统



2、STM32F1系列主流微控制器

ST意法半导体公司产品

ARM Cortex-M3内核 32位微控制器

高性能 低成本、低功耗、嵌入式应用

STM32F100

超值型

STM32F101

基本型（标准型）

入门产品，36MHz

STM32F102

USB基本型

STM32F103

增强型

同类产品性能最高，72MHz，16K-512K闪存

带有更多片内SRAM和更丰富的外设

STM32F105/107

互联型

STM32L

超低功耗型



3、ARM微处理器系列

系列	主要特征	应用领域
ARM7	32 位 RISC 处理器；指令系统与 ARM9、ARM9E 和 ARM10E 系列兼容	工业控制、Internet 设备、网络和调制解调器设备、移动电话等多种多媒体和嵌入式应用。
ARM9	基于哈佛结构，32 位 ARM9 系列微处理器执行 ARMv4T 指令集，在高性能和低功耗特性方面提供最佳的性能。	可用于PDA 等高档手持设备、MP5 播放器等数字终端、数码相机等图像处理设备及汽车电子方面。
ARM9E	支持ARM、Thumb 和 DSP 指令集，提供浮点运算协处理器，用于图像和视频处理。执行 ARMv5TEJ 指令集。	网络通信设备、移动通信设备、图像终端、海量数据存储设备、汽车的智能化设备等。
ARM10E	执行ARMv5TEJ 指令集，支持 ARM、Thumb、DSP 和 Java指令，支持实时操作系统和嵌入式操作系统。	下一代无线设备、数字消费品、成像设备、工业控制、通信和信息等领域
ARM11	执行ARMv6 指令集，针对高性能和高能效应用而设计的，增加了向量浮点单元	数字TV、机顶盒、游戏终端、汽车娱乐电子、网络设备等
Cortex	属于ARMv7 架构	"A"系列面向尖端的基于虚拟内存的操作系统和用户应用；"R" 系列针对实时系统；"M"系列对微控制器。
SecurCore	高度集成的 32 位RISC 微处理器；在软件上兼容 ARMv4 体系结构、同时采用具有 Intel 技术优点的体系结构	面向智能卡、电子商务、银行、身份识别、电子购物等信息安全设备应用领域。
XScale	Intel主要推广的一款基于 ARMv5TE 体系结构 ARM 微处理器	平板计算机、GPS 定位系统和网络产品等场合





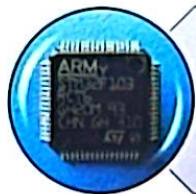
第二章 嵌入式单片机STM32硬件基础



2.1 STM32系列单片机外部结构、内部结构



2.2 STM32系列单片机开发板



2.3 最小系统设计





1、STM32F10x总线架构—相关参数

总线是各种信号线的集合，是嵌入式系统中各部件之间传送数据、地址和控制信息的公共通路。

与总线相关的参数

总线宽度：总线能同时传送的数据位数（bit），如32位

总线频率：总线的工作速度，频率越高，速度越快

总线带宽 = 总线宽度 × 总线频率 / 8 单位为MBps



总线带宽 = 总线宽度 × 总线频率 / 8 单位为MBps

【例】总线宽度为32bit，时钟频率为200MHz，若总线上每5个时钟周期传送一个32bit的字，则该总线的带宽为（ ）MB/S。

- (1) 由200MHz -> 1个时钟周期为： $1/200\text{MHz}=0.005\mu\text{s}$
- (2) 总线传输周期为： $0.005\mu\text{s} \times 5 = 0.025\mu\text{s}$
- (3) 总线宽度为：32位=4B（字节）
- (4) 总线的数据传输率（带宽）为： $4\text{B} / (0.025\mu\text{s}) = 160\text{MBps}$


$$200\text{M}/5 \times 32\text{bit} / 8\text{bit} = 160\text{MB/S}$$





2、STM32F10x总线架构—处理器结构

根据指令和数据所用的存储空间与总线形式的不同，处理器分为：

冯 诺依曼结构

- 微处理器指令和数据共用一个存储空间和一条总线
- 内核在取指时不能进行数据读/写

哈佛结构

- 微处理器指令和数据存储在不同的存储空间空间和一条总线
- 采用独立的指令总线 and 数据总线，可以同时进行取指和数据读/写操作

ARM处理器均采用哈佛结构

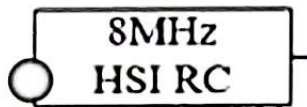




3、STM32时钟源

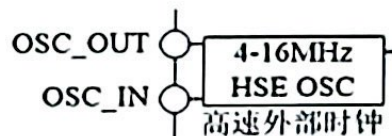
HSI

- High Speed Internal, 高速内部时钟, 由内部8MHz的RC振荡器生成, 可作为系统时钟或2分频后作为PLL输入; 特点: 时钟频率精度差, 不稳定。



HSE

- High Speed External, 高速外部时钟, 可外接一个外部时钟源, 或者通过OSC_IN和OSC_OUT引脚外接晶振, 允许外接的晶振频率范围为4~16MHz, 通常使用8MHz。特点: 精度高, 稳定。



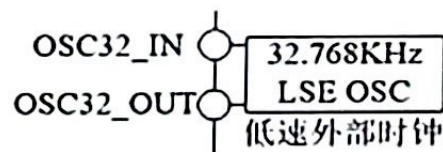
LSI

- Low Speed Internal, 低速内部时钟, 由内部RC振荡器产生, 频率约40kHz, 主要为独立看门狗 (IWDG) 和自动唤醒单元提供时钟。



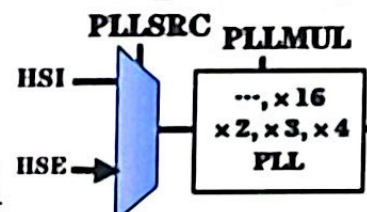
LSE

- Low Speed External, 低速外部时钟, 通过OSC32_IN和OSC32_OUT引脚外接频率为32.768kHz的晶振, 主要为RTC (Real-Time Clock, 实时时钟部件) 提供低速高精度的时钟源。



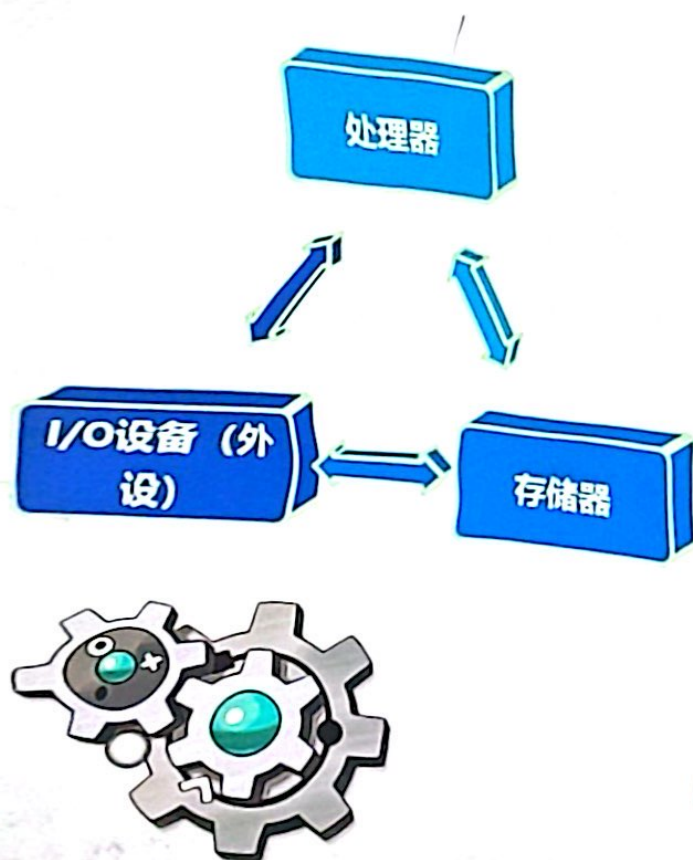
PLL

- Phase Locked Loop, 锁相环, 保证外部输入时钟信号与内部振荡信号同步 (频率和相位相同), 以保证输出频率的稳定。另一方面, 也可用于倍频HSI或HSE。





4、STM32处理器内部存储器结构



- STM32总的地址空间大小为4GB，用0x0000 0000H~0xFFFF FFFF来表示
- 将4GB大小的空间划分为：
 - ✓ Flash程序存储器区
 - ✓ SRAM静态数据存储器区
 - ✓ 片上外设区

$$2^{32} = 2^2 \times 2^{10} \times 2^{10} \times 2^{10} = 4 \times 1024 \times 1024 \times 1024 = 4GB$$





第三章 嵌入式单片机STM32软件开发基础

日 积 月 累
勤 学 苦 练
熟 能 生 巧

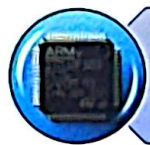




第四章 STM32单片机的通用功能输入输出GPIO



4.1 STM32F103的IO端口的组成及功能



4.2 GPIO常用库函数



4.3 工程框架



4.4 GPIO使用流程及应用设计实例



1. GPIO工作模式





2. 几种常用的库函数

1、void GPIO_Init 功能：对GPIOx端口指定的一些引脚进行初始化

void GPIO_Init (GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct)

```
11  GPIO_InitTypeDef  GPIO_InitStructure;  
12  
13  GPIO_InitStructure.GPIO_Pin =    GPIO_Pin_2 ;  
14  GPIO_InitStructure.GPIO_Mode =  GPIO_Mode_Out_PP;  
15  GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;  
16  GPIO_Init(GPIOD, &GPIO_InitStructure);
```



5、void **GPIO_WriteBit**(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, BitAction BitVal)功能：对GPIOx端口的GPIO_Pin引脚写1/0，即输出1或0。

枚举类型“BitAction”的定义：

```
typedef enum
{
    Bit_RESET = 0,
    Bit_SET    // = 1
}BitAction;
```

6、void **GPIO_Write** (GPIO_TypeDef *GPIOx, uint16_t PortVal)

功能：对GPIOx端口写PortVal值，即从GPIOx端口输出PortVal值

○



7、u16 **GPIO_ReadInputData**(GPIO_TypeDef* GPIOx)

功能：读取GPIOx端口输入的16位数据值

8、u8 **GPIO_ReadInputDataBit**

(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)

功能：读取GPIOx端口的GPIO_Pin引脚值，返回值在最低位。





1.

```
temp1=GPIO_ReadInputDataBit(GPIOA,GPIO_Pin_8);
```

```
temp2=GPIO_ReadInputDataBit(GPIOA,GPIO_Pin_9);
```

```
if (temp1==1)  GPIO_Write(GPIOC,0xFF00);
```

```
if (temp2==1)  GPIO_Write(GPIOC,0x00FF);
```

该程序功能为:





1. 相关概念

- 1) 单片机与外设交换数据的三种方式
- 2) 中断的相关概念：中断、中断源、中断优先级……



2. 中断的优势

为什么设置中断

- ◆ 提高CPU工作效率
- ◆ 具有实时处理功能
- ◆ 具有故障处理功能
- ◆ 实现分时操作

在程序设计时，采用中断方式主要有哪些优势？





模块化

简要叙述模块化编程的优点。

程序更加清晰便于程序改动

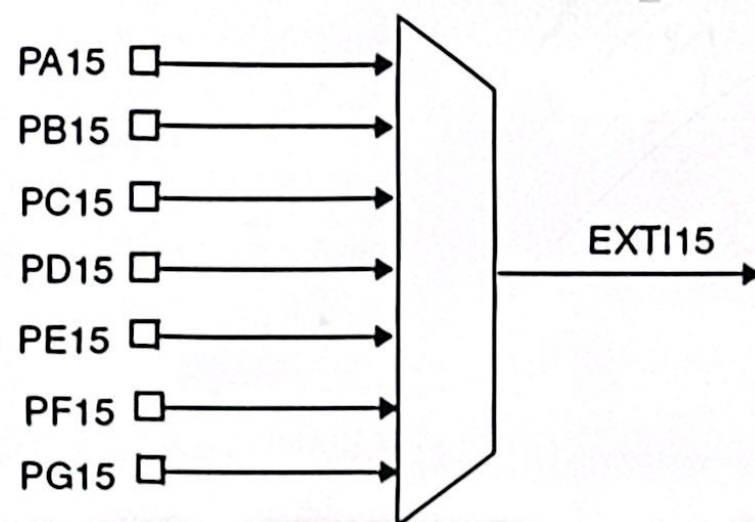
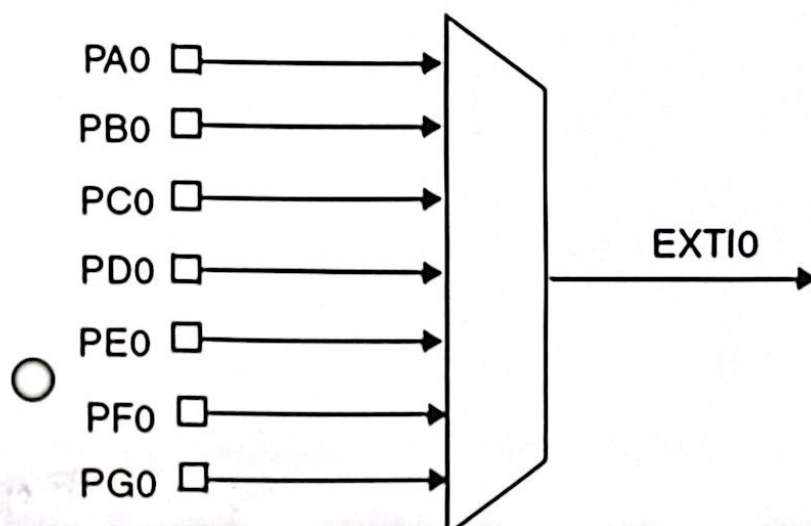
- 不同功能模块设计成小耦合度模块，降低文件之间的依赖关系。
- 提高源代码的复用率，降低代码占用的空间，提高程序可靠性。
- 提高程序的可维修性，延长程序生命周期。





(5) GPIO外部输入中断

GPIO的中断是以**组**为单位的，同组的外部中断**公用一条外部中断线**。



3. 中断源及中断向量

STM32F103中断系统提供10个系统异常和60个可屏蔽中断源，具有16个中断优先级。



STM32F103可屏蔽中断源

由NVIC和相应的外设控制

由NVIC和EXTI控制

外部中断 (GPIO)

外设中断

定时器中断

串口中断

直接内存访问中断

模数转换中断

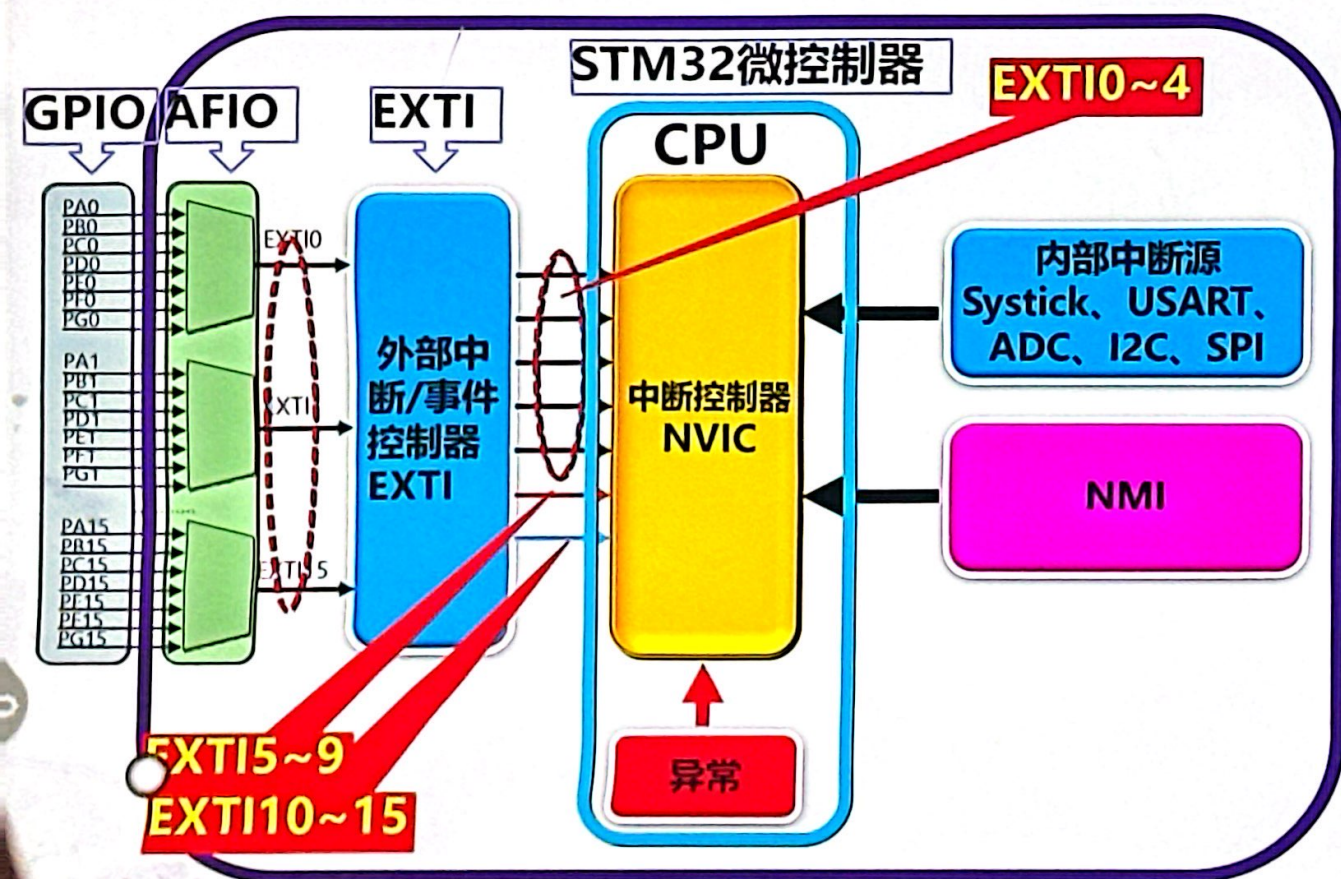
集成电路总线中断

串行外设接口中断





3. 中断源及中断向量



1 嵌套中断向量控制器NVIC

在Cortex-M3中内建的中断控制器，中断发生时，自动获得服务例程入口地址并直接调用。

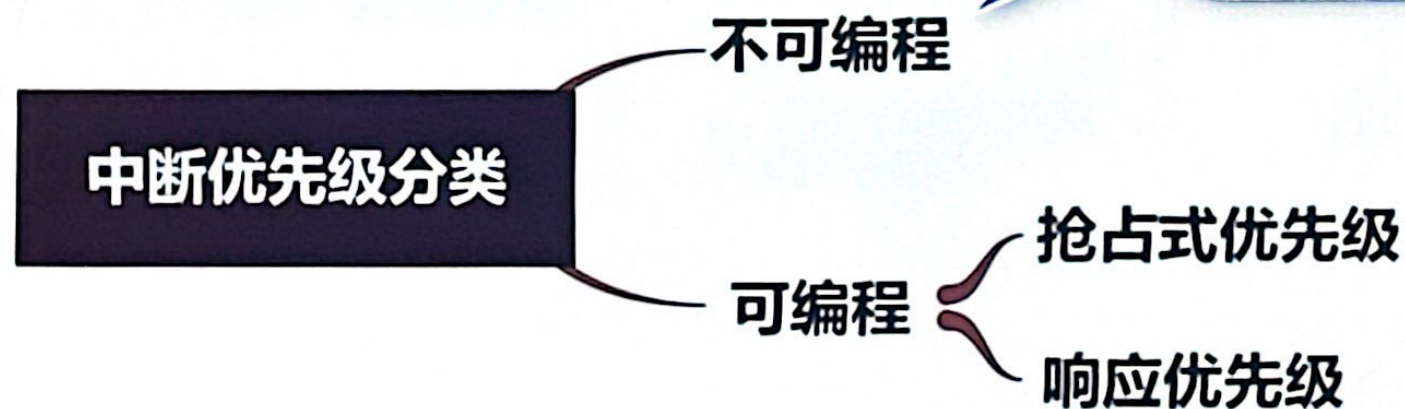
2 外部中断/事件控制器EXTI

由19个产生中断/事件要求的边沿检测器组成。每个输入线可以独立地配置输入类型（脉冲或挂起）和对应的事件触发方式（上升沿或下降沿或者双边沿都触发）。



4. STM32F103中断优先级

中断优先级规则





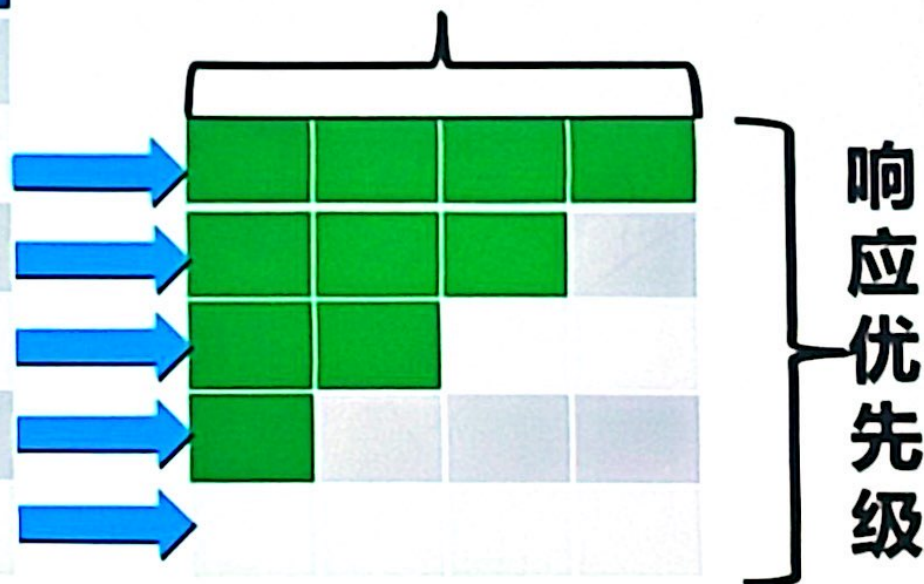
4. STM32F103中断优先级

优先级设置分组

使用中断时，
首先应指明使用哪一组优先级分级方案。

优先级 组别	抢占式优先级		响应式优先级	
	位数	级数	位数	级数
4组	4	2^4	0	2^0
3组	3	2^3	1	2^1
2组	2	2^2	2	2^2
1组	1	2^1	3	2^3
0组	0	2^0	4	2^4

抢占优先级





5. 中断处理

▪ 1 中断请求和优先级

- 如果系统中存在多个中断源，处理器要先对当前中断的优先级进行判断。

多个中断请求同时到达时，先响应优先级高的中断。

- 如果它们的抢占优先级相同，则先处理响应优先级高的中断。



5. 中断处理

- 例：有三个中断向量的抢占优先级和响应优先级配置如表中所示，问题如下：如果内核正在执行C的中断服务程序，是否会被中断A和中断B打断？为什么？

有三个中断向量：		
中断向量	抢占优先级	响应优先级
A	0	0
B	1	0
C	1	1



6. 中断应用设计

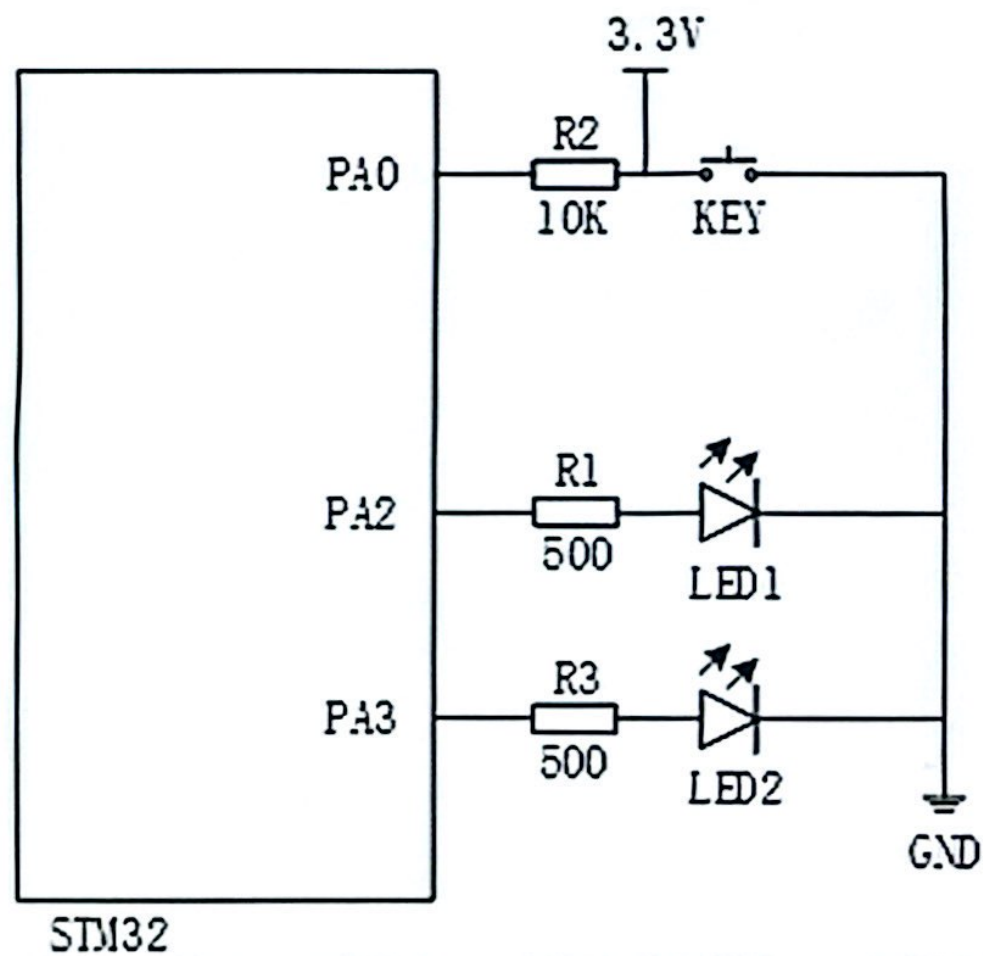
实现功能：上电后发光二极管LED1灭，LED2亮，由按键KEY控制两个LED灯轮流亮：按键KEY第1次按下时LED1亮、LED2灭，按键KEY第2次按下时LED1灭、LED2亮，按键KEY第3次按下时LED1亮、LED2灭，……以此类推。

- (1) 绘制硬件原理图简图；（GPIO口使用PA0、PA2、PA3）
- (2) 绘制流程图；（包括主程序流程图和中断程序流程图）
- (3) 编写主程序中while循环的程序，编写中断程序。（LED灯亮灭程序默认已编好，如LED1_On（）可在程序中直接使用。中断程序中使用检测中断线请求EXTI_GetITStatus，清除中断挂起EXTI_ClearITPendingBit。）



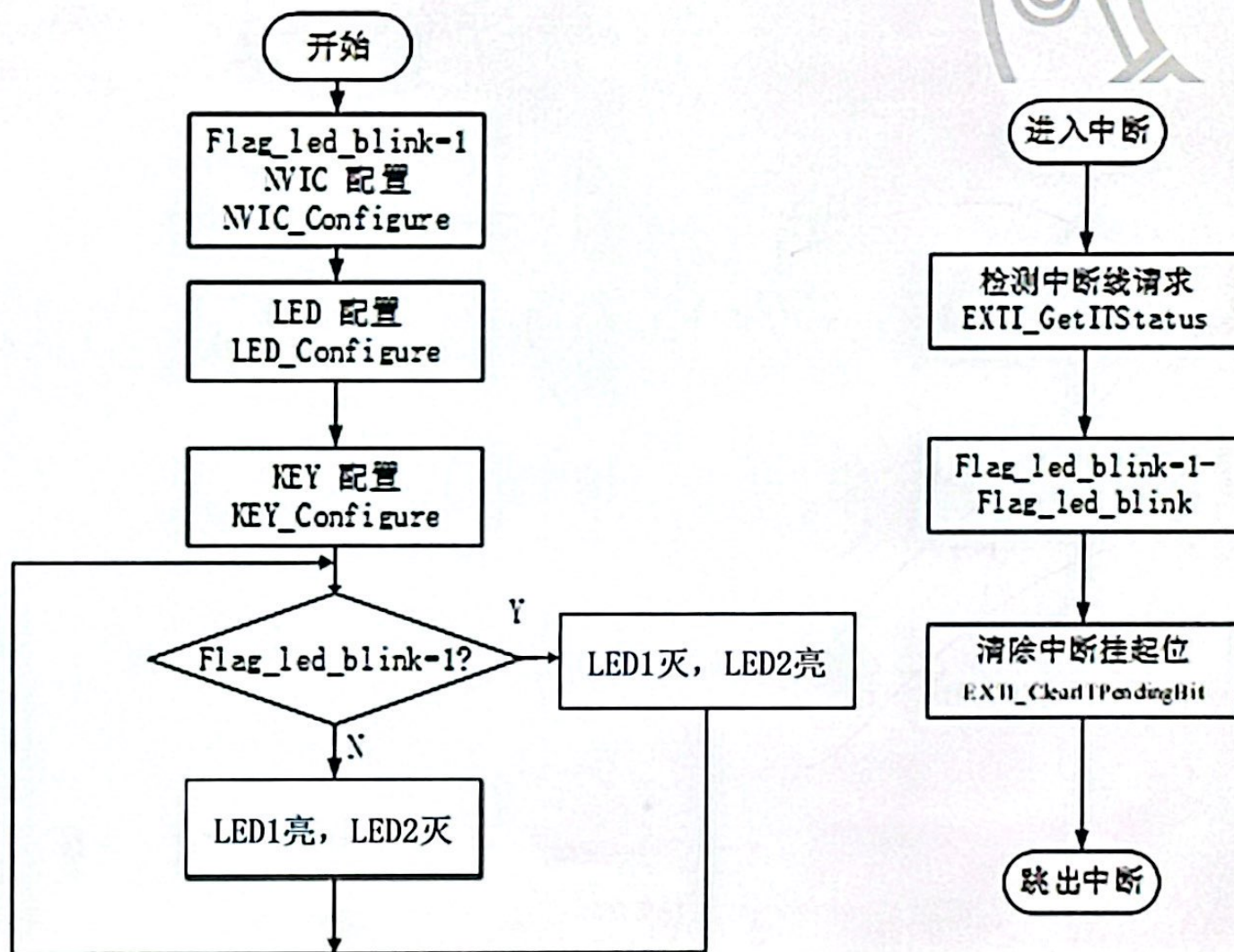
6. 中断应用设计

(1) 绘制硬件原理图简图；（GPIO口使用PA0、PA2、PA3）



6. 中断应用设计

(2) 绘制流程图;
(包括主程序流程图
和中断程序流程图)



6. 中断应用设计

(3) 编写主程序中while循环的程序，编写中断程序。

```
while (1)
{
    if(flag_led_blink )
    {
        LED1_Off();
        LED2_On();
    }
    else
    {
        LED1_On();
        LED2_Off();
    }
}
```

```
void EXTI0_IRQHandler(void)
{
    if(EXTI_GetITStatus(EXTI_Line0)!=RESET)
    {
        flag_led_blink = 1 - flag_led_blink ;
        EXTI_ClearITPendingBit(EXTI_Line0);
    }
}
```



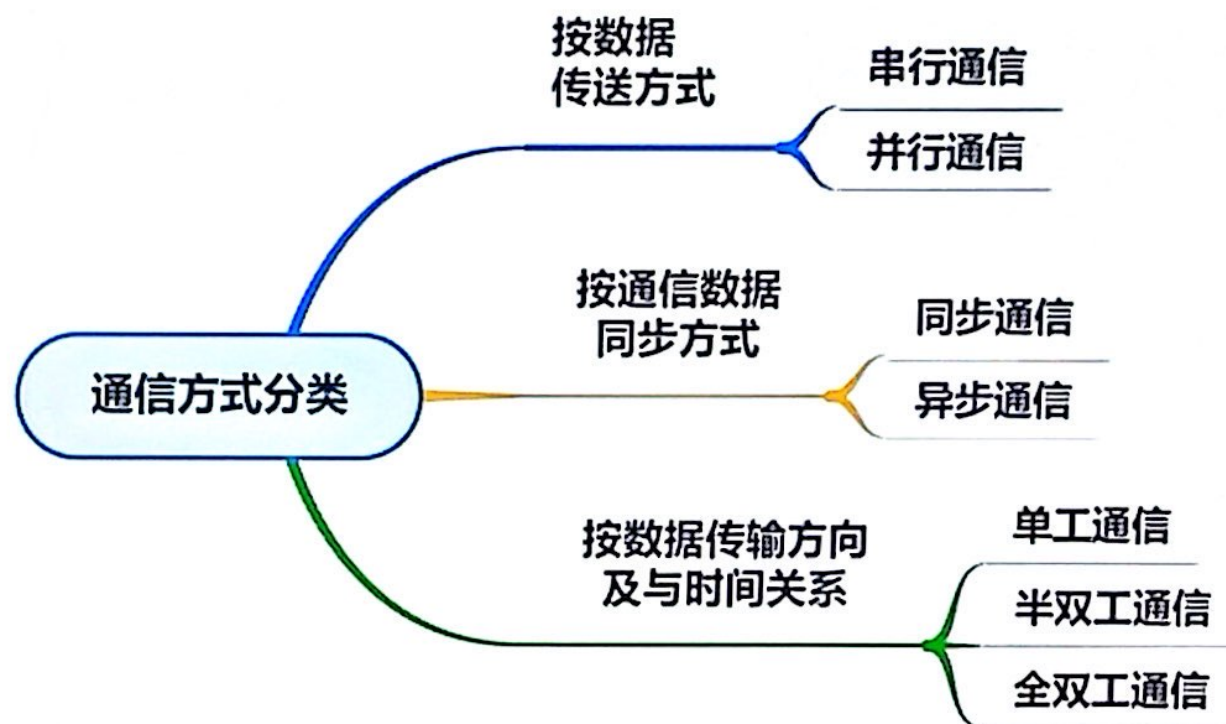


第六章 STM32通用同步/异步通信

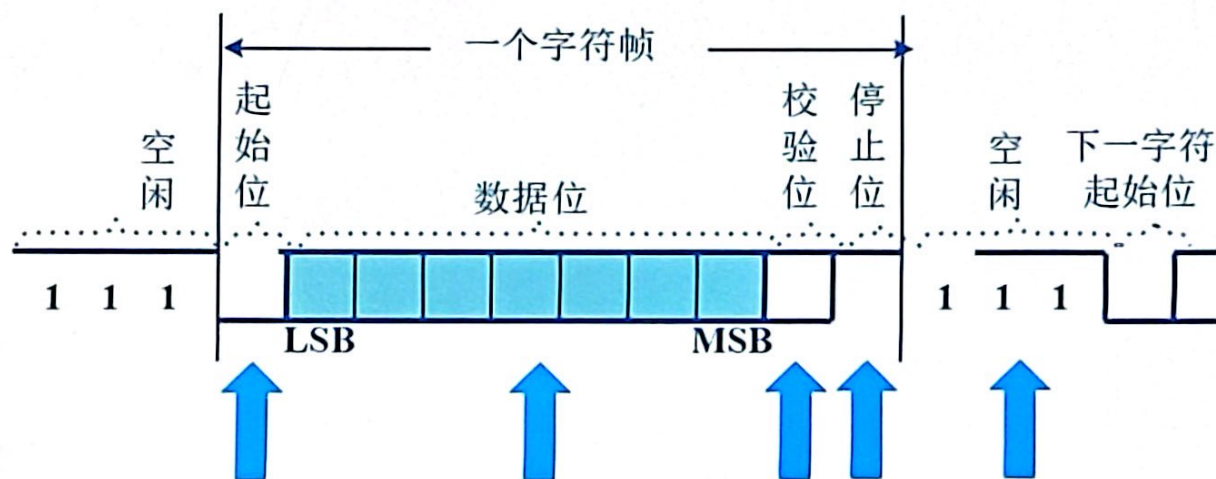
- 6.1 串行通信简介
- 6.2 STM32的USART的结构及工作方式
- 6.3 USART常用库函数
- 6.4 USART使用流程
- 6.5 USART应用设计实例
- 6.6 串行通信接口抗干扰设计



1. 通信方式分类



2. 数据帧格式



起始位：占一位，位于数据帧的开头，以逻辑“0”表示传输数据的开始。

数据位：要发送的数据，数据长度可以是5~8位。

校验位：占一位，用于检测数据是否有效。

停止位：一帧传送结束的标志，根据实际情况定，可以是1、1.5或2位。

空闲位：数据传输完毕，用“1”表示当前线路上没有数据传输。





3. STM32的USART简介

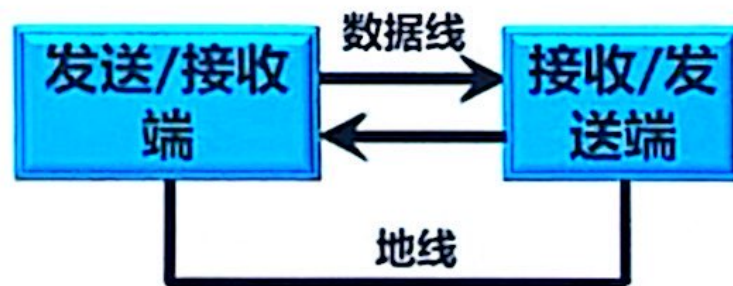
STM32有3-5个的全双工的异步串行通信接口USART，可实现设备之间的串行数据传输。

USART (Universal Synchronous Asynchronous Receiver and Transmitter, 通用同步异步收发器) 是一个**串行通信**设备，可以灵活地与外部设备进行**全双工**数据交换。

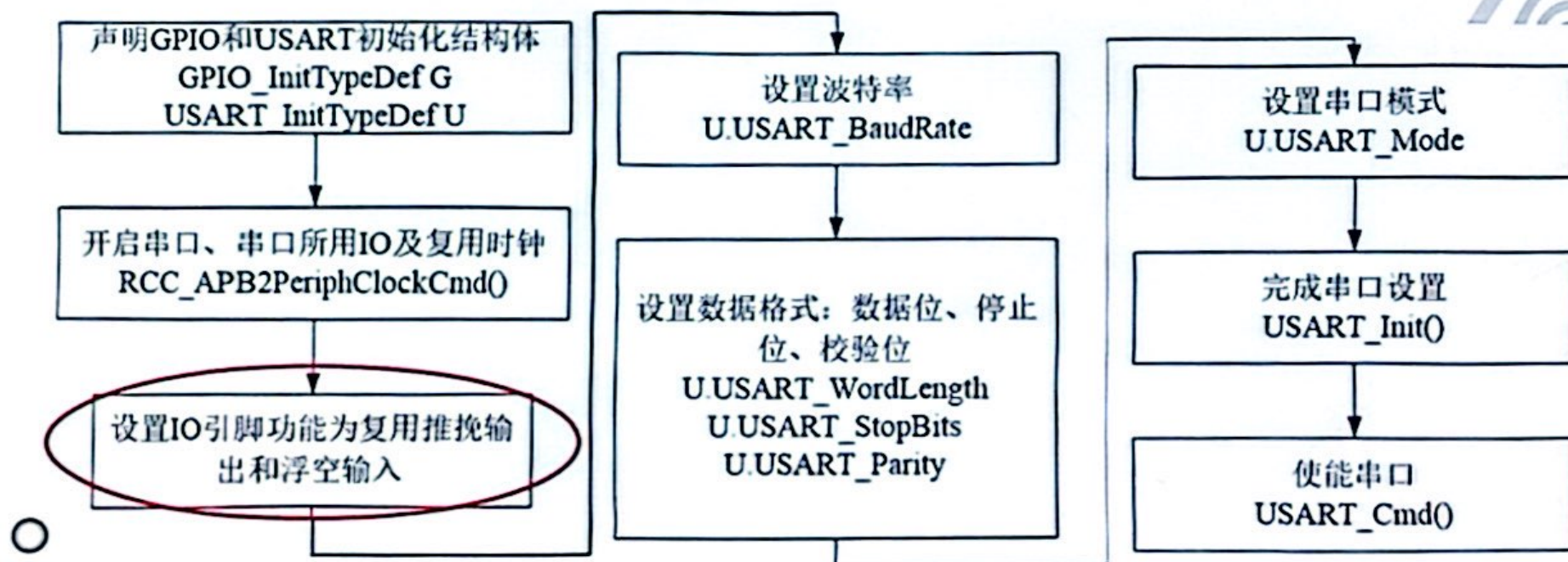
UART(Universal Asynchronous Receiver and Transmitter) 跟 USART 不一样的是：它是在 USART 基础上裁剪掉了**同步通信**功能，只有**异步通信**。



4. 三线制



6、串口基本配置流程





6、串口基本配置流程

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;  
GPIO_Init(GPIOA, &GPIO_InitStructure);  
  
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;  
GPIO_Init(GPIOA, &GPIO_InitStructure);
```

设置IO引脚功能为复用推挽输出和浮空输入

位、校验位
U USART_WordLength
U USART_StopBits
U USART_Parity

使能串口
USART_Cmd()

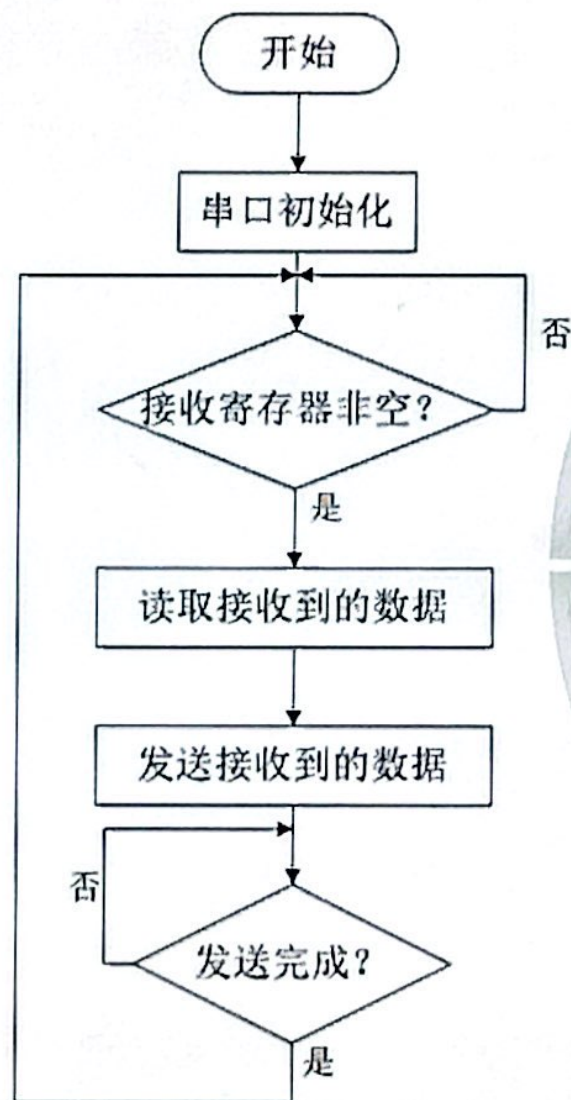




7、例1 收发信息

功能需求分析:

- 1、接收上位机发送过来的数据;
- 2、将接收到的数据再回传给上位机;



7、例1 收发信息

```
int main(void)
{
    uint16_t dat;
    USART1_Configure();
    while(1)
    {
        if(USART_GetFlagStatus(USART1,USART_FLAG_RXNE)==SET)
        {
            dat = USART_ReceiveData(USART1); //
            USART_SendData(USART1, dat ); //
            while(USART_GetFlagStatus(USART1,USART_FLAG_TC)!=SET);
        }
    }
}
```



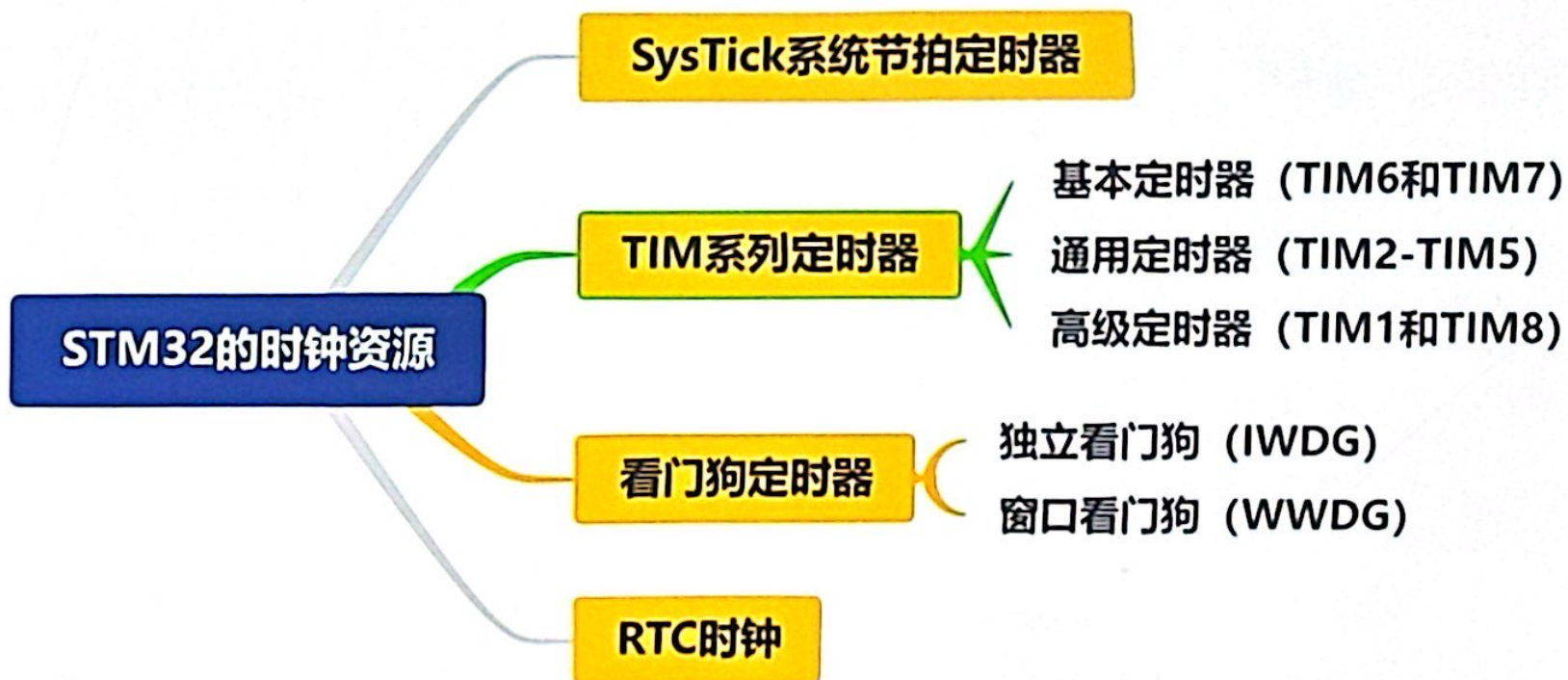


第七章 STM32通用定时器

- 7.1 STM32定时/计数器概述
- 7.2 STM32通用定时器的结构
- 7.3 STM32通用定时器的功能
- 7.4 通用定时器的常用库函数
- 7.5 通用定时器的使用流程
- 7.6 通用定时器的应用设计实例



1、定时器的类型





1、定时器的类型 - TIM系列定时器

STM32F103具有8个TIM系列定时器，其中，

- TIM1和TIM8为高级控制定时器
- TIM2~TIM5为通用定时器，其结构和工作原理相同
- TIM6和TIM7为基本定时器

相对于传统的51单片机的定时器而言，STM32F103的定时器功能更加完善和复杂。



2、通用定时器功能

通用定时器的基本功能是**定时和计数**。

当可编程定时/计数器的时钟源来自**内部系统时钟**时
可以完成精密**定时**；

当时钟源来自**外部信号**时可完成外部信号**计数**。



第九章 STM32的模/数转换器

- 9.1 STM32应用系统简介
- 9.2 STM32的ADC结构
- 9.3 ADC的工作模式
- 9.4 ADC的常用库函数
- 9.5 ADC的使用流程
- 9.6 ADC的应用设计实例



1、STM32的ADC结构

- STM32拥有1~3个ADC (STM32F101/102系列只有1个ADC, STM32F103系列最少都拥有2个ADC), 这些ADC可以独立使用, 也可以使用双重模式 (提高采样率)。
- STM32的ADC是12位逐次逼近型的模拟数字转换器。它有18个通道, 可测量16个外部和2个内部信号源 (温度传感器、内部参考电压)。





2、分辨率

表征了ADC对输入模拟信号最小变化的分辨能力。

分辨率与ADC的位数有关，为满刻度电压与 2^n 的比值， n 为ADC的位数。常见的位数主要包括：6位、8位、10位、12位、16位。



(2) 功率放大驱动

经数/模转换得到的模拟电压或电流控制信号,不能满足控制对象的功率要求,必须经功率放大,驱动外部系统。

(3) 干扰信号防治

后向输出通道接近控制对象,工作环境相对恶劣,驱动系统会通过信号通道、电源以及空间电磁场对单片机应用系统造成电磁干扰,另外还会出现机械干扰,因此通常采用信号隔离、电源隔离和大功率开关实现过零切换等方法进行干扰信号防治。

处理的结果可以以开关量、数字量和频率量输出。

9.1.2 ADC 的性能指标

无论是分析或设计 ADC 的接口电路,还是选购 ADC 芯片,都会涉及有关性能指标的术语。因此,弄清 ADC 的基本概念以及一些经常出现的性能指标术语的确切含义是十分必要的。

1. 分辨率 (Resolution)

对于 ADC 来说,分辨率表示输出数字量变化一个相邻数据码所需输入模拟电压的变化量,反映了 ADC 对输入模拟信号最小变化的分辨能力。

ADC 的分辨率定义为满刻度电压与 2^n 的比值,其中 n 为 ADC 的位数。例如,12 位 ADC 能够分辨出满刻度 $1/2^{12}$ (或满刻度 0.024%) 的输入电压的变化。一个 10V 满刻度的 12 位 ADC 能够分辨输入电压变化的最小值为 2.4mV。表 9-1 列出了不同位数 n 与分辨率的关系。

表 9-1 ADC 的分辨率与位数的关系

位数 n	分辨率	
	分数	满刻度百分数 (近似)
8	1/256	0.4
10	1/1024	0.1
11	1/2048	0.05
12	1/4096	0.024
14	1/16384	0.006
15	1/32768	0.003
16	1/65536	0.0015

2. 量化误差 (Quantizing Error)

量化误差是由 ADC 的有限分辨率而引起的误差。在不计其他误差的情况下,一个有限分辨率 ADC 的阶梯状转移特性曲线与无限分辨率 ADC 转移特性曲线 (直线) 之间的最大偏差,称之为量化误差。如果在零刻度处适当偏移 $\frac{1}{2}$ LSB,则量化误差为 $\pm \frac{1}{2}$ LSB。如果没有加入偏移量,则量化误差为 -1 LSB。

3. 偏移误差 (Offset Error)



大争之世，非优即汰！

崛起之时，不进则退！





豆包AI



CS 扫描全能王
3亿人都在用的扫描App